

Lecture 05 — The while loop — the program that insists

Algorithmic Thinking · Week 3, Session 5 — Tuesday 6 October 2026,
17:35–18:55 (Quick exam 2 in the first 20 minutes)

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview	1
Opener — Why the internet does not collapse (10 min)	1
Notes	2
Stretch — if the session was easy	6
After the session	6

Objective. Write a loop that repeats while a condition holds, name the line that makes it stop, and prove it does.

Overview

Quick exam 2 takes the first twenty minutes. Then the program learns to **insist**.

Part C of the exam you just sat said: *if the consumption is negative, display a message and stop*. Stop? A real program does not stop. It says “try again” — and keeps saying it until the answer is acceptable. That is one line of Python we did not have until today, and it is the whole lecture.

One idea: **while runs a block again and again, for as long as its condition is True**. Everything else — counting, adding up, reading until a sentinel, the loop that never ends — is that idea with a different condition. The skill is not writing the loop. The skill is knowing, before you run it, **which line makes it stop**.

Opener — Why the internet does not collapse (10 min)

Algorithms to Live By, chapter 10, *Networking*. Thirty laptops in this room share one Wi-Fi channel. Two of them send at the same instant — the signals collide and both messages are garbage. What should a laptop do when its message collides?

Take three answers. “Send it again immediately” — both laptops do that, they collide again, forever. “Wait a bit” — how long? If everyone waits the same fixed time they collide again on schedule. The answer that has run every Ethernet and Wi-Fi network since 1973 is **exponential backoff**:

After the first collision, wait a random time between 0 and 1 slot. After the second, between 0 and 3. Then 0 to 7, 0 to 15 — **doubling the range every time**. Give up after 16 tries.

Three things make it an algorithm worth stealing. It **repeats while** a condition holds (the message is not through yet). It has a **step** that changes every repetition (the waiting range doubles). And it has an **exit you can prove**: either the message gets through or the counter reaches 16 — nothing can keep it running forever. The book's point is that the same shape is how you should treat a friend who cancels: don't cut them off at the first miss, don't tolerate forever — double the gap each time.

Your turn, 3 minutes, on paper: write *send with backoff* in our notation. INPUT: (the message), OUTPUT: (sent, or gave up), numbered steps. You will need a counter for the attempts and a variable for the waiting range. **Circle the step that decides whether to go round again.** Then the pivot, out loud: *that circled step is the only new word Python learns today.*

Notes

The line we could not write

Open 01_ask_again.py. On the exam, a negative consumption got a message and the program ended. Now:

```
cubic_meters = float(input("Consumption (m3): "))
while cubic_meters < 0:
    print("Consumption cannot be negative. Try again.")
    cubic_meters = float(input("Consumption (m3): "))
print("Thank you. Billing", cubic_meters, "m3.")
```

Run it; type -3, -1, 12. Read the new word out loud: **while** the consumption is negative, do this block. Not *if* — *while*. The difference between the two keywords is one arrow: after the block, while **goes back to the test**. *if* goes on.

Everything you know about conditions from last week transfers unchanged: it is an expression with a True/False value, == compares, the block is what is indented.

Start, test, step

Open 02_count_to_five.py:

```
count = 1                # start
while count <= 5:         # test
    print(count)
    count = count + 1     # step
print("Done. count is now", count)
```

Predict the last line before running. Most of the room says 5. It prints **6**: the loop ends when the test *fails*, and the test fails when count has already become 6. The variable you count with is always one step past the last thing you printed. Remember this on Friday — it is where half of all off-by-one bugs come from.

Three parts, always: **start** (the variable exists and has a value *before* the while), **test** (the condition on the while line), **step** (the line inside the block that changes the variable the test looks at). Find all three in 01_ask_again.py. The step is the input() — the user supplies the change.

Reading until a sentinel

Open 03_sentinel_average.py. Week 1's problem — the average speed — with the number of measurements unknown:

```
total = 0
count = 0
speed = float(input("Speed (0 to finish): "))
while speed != 0:
    total = total + speed
    count = count + 1
    speed = float(input("Speed (0 to finish): "))
if count > 0:
    print("Measurements:", count, "Average:", total / count)
else:
    print("No measurements.")
```

Run with 100, 90, 140, 0 → 3 measurements, average 110.0. The 0 is the **sentinel**: a value the user types to say *I am done*, and which therefore cannot be a real measurement. Notice the shape — the first input() is *before* the loop, the next one is the *last line inside* it. That is the sentinel idiom; copy it exactly until it is a reflex.

Two more things to see. total and count start at 0 before the loop and grow by one step per turn — that is the **accumulator pattern**, and you will write it a hundred times. And the if count > 0 after the loop: the user may type 0 first. A while block can run **zero times**, and total / count would then be 0 / 0. The margin note from week 1 has come back, and this time you know what to do with it.

The loop that never ends

Open 04_forever.py. Read it before running — then run it, and press **Ctrl+C** when you have seen enough:

```
n = 10
while n > 0:
    print("n is", n)
    # n = n - 1
```

The test looks at n. Nothing inside the block changes n. So the test gives the same answer forever. This is the only kind of bug in this course that does not produce an error message and does not produce wrong output — it produces *no end*. Thonny's stop button, or Ctrl+C in a shell, is how you get out. Every one of you will write one this week. That is fine; what matters is recognising it in two seconds: **find the test, find the step, see that the step is missing**.

Now the positive version. How do you *know* a loop stops? Name the quantity that moves toward the exit on every single repetition. In 02, count goes up by one toward 6. In 03, every turn consumes one of the user's finite patience — the user *will* type 0. In the backoff algorithm, the attempt counter goes up toward 16. If you cannot name the quantity, you do not know your loop stops — you are hoping.

Counting with while — and the step order

Open 05_sum_to_n.py:

```
n = int(input("Add 1 to what number? "))
total = 0
i = 1
while i <= n:
    total = total + i
    i = i + 1
print("1 + ... +", n, "=", total)
```

10 → 55. 1 → 1. 0 → 0 — the block never runs, and that is correct. -3 → 0 too; is *that* correct? (Specification question. The program is fine; the spec did not say.)

Now swap the two lines inside the block — step first, then add. Predict, then run 10. You get 65: i was already 2 when it was first added, and 11 was added at the end. **The order of the step inside the block changes what the loop does.** This is the second source of off-by-one bugs, and Friday is built around it.

Three accumulators at once

Open 06_max_so_far.py — the same loop as 03, now with a maximum and an ignored-negative branch:

```
maximum = None
...
if speed < 0:
    print("Negative speed ignored.")
else:
    total = total + speed
    count = count + 1
    if maximum is None or speed > maximum:
        maximum = speed
```

maximum = None says *no maximum yet*. The first real speed becomes the maximum; after that, only a bigger one replaces it. Read the or: if there is no maximum yet, take this one; else compare. 100 -5 140 90 0 → count 3, average 110.0, max 140.0.

Last week's if lives *inside* this week's while, four spaces deeper. Blocks nest. Read the indentation, not your intention.

A loop inside a function

Open 07_positive_sum_function.py — Friday's lab shape, exactly:

```
def positive_sum(how_many):
    total = 0
    asked = 0
    while asked < how_many:
        value = float(input("Number: "))
        if value > 0:
            total = total + value
```

```

        asked = asked + 1
    return total

def main():
    how_many = int(input("How many numbers? "))
    print("Sum of the positive ones:", positive_sum(how_many))

main()

```

Nothing new, and that is the point: a while inside a def works exactly like an if inside a def. The function returns, main prints. The input() *inside* the function is the academia's design for this one (the function asks for its own numbers) — we accept it here because the lab says so, but notice the cost: positive_sum(3) cannot be tested from the shell without typing three numbers. On Friday we talk about whether that is a price worth paying.

Shell checks after running: positive_sum(0) → 0 **without asking anything** — the block ran zero times. That is the test that catches while asked <= how_many.

Failure-first: three loops that run and are wrong

Open 08_three_bugs.py. Predict each before running; keep Ctrl+C ready.

1. **The step outside the block.** i = i + 1 at the margin, not indented: it belongs to the program, not to the loop. The loop prints A 1 forever. Indentation is the block — week 2's rule, now with consequences you can watch.
2. **Off by one.** i = 0 · while i < 5 · i = i + 1 · print(i) prints 1 to 5. Move the print above the step and it prints 0 to 4. Same five lines, different program. Which one did the specification ask for? *Read the spec.*
3. **The sentinel is also data.** If 0 ends the loop, nobody can enter a real 0. Choose a sentinel outside the data's range — -1 for speeds — or the design is wrong before a single line of Python exists.

In-class activity 2 — Speeds, until you stop (*individual, 20 min, graded — the second of the four in-class activities, 4 % of the course*)

Week 1's average, week 2's checks, today's loop. One program.

1. Read speeds (km/h) from the user, one at a time, **until the user enters 0**.
2. A **negative** speed is not a measurement: display Negative speed ignored. and keep going.
3. When the user enters 0, display: how many measurements were taken, the **average** speed and the **maximum** speed.
4. If **no** measurements were taken, display No measurements. and nothing else.

Algorithm first, on paper — INPUT:, OUTPUT:, numbered steps, with the WHILE line and its IF lines written out. **Circle the step that makes the loop stop.** Then the program in the exam format (name in a # comment on line 1, algorithm in """). Then the tests, under the algorithm:

you type	expected
100 90 140 0	3 measurements · average 110.0 · maximum 140.0
0	No measurements.
-5 60 0	Negative speed ignored. · 1 measurement · average 60.0 · maximum 60.0
80 80 0	2 measurements · average 80.0 · maximum 80.0

The 0-first row is the one that catches `total / count`. The 80 80 row catches a maximum written with `>=` *and* with `>` — both are right, and you should be able to say why.

Submit: photo of the algorithm + `A_Matricula_ic2.py` → Canvas → *In-class activity 2: while loops — speeds, until you stop* — by **Sunday 11 October, 23:59**, but the work happens now, in the room.

Rubric: algorithm with input/output, the WHILE and its IFs stated, and the stopping step circled 40 % · program runs and passes all four tests 40 % · the zero-measurements case and the ignored negative handled on purpose 20 %.

Finished early? Make 0 a valid measurement by choosing a different sentinel — and write, in one sentence under the algorithm, why your new sentinel can never be real data.

Stretch — if the session was easy

Optional, ungraded, harder than they look. Bring them to asesorías or leave them in the week’s evidence box.

1. **Prove it stops.** For each of 02, 03, 05 and 07, write the one quantity that moves toward the exit on every repetition, and the value at which the loop ends. Then write a loop whose test *does* change every turn and which still never ends. (Hint: a variable that alternates.) Say why your termination argument fails for it.
2. **Backoff, in Python.** Implement the opener: attempts from 1 to 16, a waiting range that doubles (`limit = limit * 2`), and `random.randint(0, limit)` as the wait. Decide “sent” with `random.random() < 0.3`. Print each attempt. Run it ten times: what is the largest number of attempts you saw, and what is the largest wait? Then answer: why *random*, and not just “double the wait”?
3. **The 65.** In `05_sum_to_n.py`, swapping the two lines in the block turns 55 into 65 for $n = 10$. Without running, write the formula for the wrong answer as a function of n , and check it for $n = 1, 2, 3$. Then design one test value that distinguishes the two programs *and* would have caught the bug in a program that only printed the final sum.

After the session

Homework 4 — assignments/hw-04-while.md. Due Sunday 11 October, 23:59, on Canvas. One file, two functions and a main: the Collatz count and the digit reversal.

Both are while loops whose exit you must be able to prove. Algorithm on paper, shell tests written down.

Also before Friday:

1. **Levelling module — all ten lessons by Thursday 8 October, 23:59.** The module closes. The **final diagnostic (1 %)** is **Friday 9 October, 16:30, in the lab** — first twenty minutes, bring the laptop. Lessons 9 and 10 (*Flowcharts with repetitions, Problem solving*) are today's lecture drawn as diamonds and arrows; do them tonight.
2. **Lab 3 — Friday 16:30**, with Luis: the final diagnostic, then the academia's while lab (two functions with loops — `C1_matricula.py`, `C2_matricula.py`). Same lab as every group.
3. **Friday's session: four exercises plus main, all loops, all with the off-by-one lurking.** Reread *Start, test, step* before you come.
4. **Quick exam 3 is Tuesday 13 October**, first 20 minutes: the while loop. Read / predict / fix, and one loop to write.